μPoly: a First Microgesture Toolkit

ADRIEN CHAFFANGEON CAILLET*, Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, France
AURÉLIEN CONIL*, Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, France
VINCENT LAMBERT, LIG, France
CHARLES BAILLY, Immersion, France
ALIX GOGUEY, Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, France
LAURENCE NIGAY, Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, France

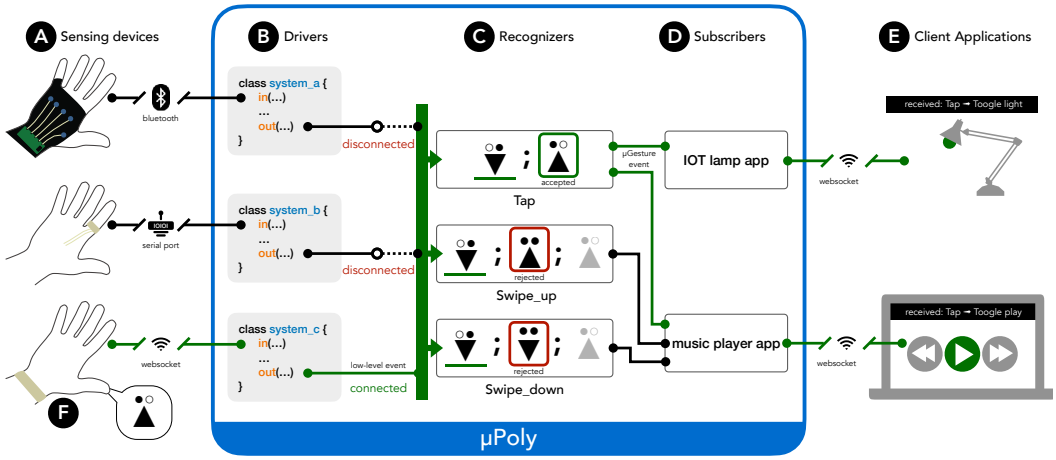

Fig. 1. Overview of μPoly. A) Sensing devices can be connected to μPoly with different communication protocols. B) Designated drivers transform the raw data from the sensing devices into low-level events, i.e. an *elementary* microgesture described with the μGlyph standard. C) These events are then fed into recognizers. Each recognizer is a μGlyph description implemented as a state machine. D) Recognized events are then transmitted to subscribers connected to E) client applications. F) In the example, only the third sensing device is connected. A release microgesture ( ) is performed. This microgesture triggers the propagation in green resulting in turning on the light and music.

*Both authors contributed equally to this research.

Authors' Contact Information: Adrien Chaffangeon Caillet, Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, Grenoble, France, chaffana@univ-grenoble-alpes.fr; Aurélien Conil, Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, Grenoble, France, conil.a@hotmail.fr; Vincent Lambert, Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, LIG, Grenoble, France, vincent.lambert@univ-grenoble-alpes.fr; Charles Bailly, Immersion, Bordeaux, France, charles.bailly@immersion.fr; Alix Goguey, Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, Grenoble, France, alix.goguey@univ-grenoble-alpes.fr; Laurence Nigay, Univ. Grenoble Alpes, CNRS, Grenoble INP, LIG, Grenoble, France, laurence.nigay@univ-grenoble-alpes.fr.

ACM 2573-0142/2026/6-ARTEICS004
https://doi.org/10.1145/3815367

With the rapid development of microgesture research, numerous recognition devices with different sensors (e.g. force-sensitive, IMU), form factors (e.g. glove, ring) and recognition algorithms (e.g. ad-hoc, random forest) have become available. With this increasing number and diversity of recognition devices, it is important to be able to easily switch between devices and/or modify the set of recognized microgestures. To address these challenges, we propose µPoly, a first microgesture toolkit based on the µGlyph microgesture notation. µPoly takes as input the µGlyph descriptions of elementary microgestures. These inputs are combined to recognize complex microgestures described using µGlyph. µPoly therefore offers a standard for microgesture events. The design of µPoly is based on properties taken from the toolkit literature. To highlight these properties: we present (1) how practitioners are using the toolkit in four scenarios, and report on user testing and (2) how applications are being developed with µPoly.

CCS Concepts: • **Human-centered computing → User interface toolkits**.

Additional Key Words and Phrases: Microgesture, Toolkit, Microgesture recognition, Gesture detection

ACM Reference Format:
Adrien Chaffangeon Caillet, Aurélien Conil, Vincent Lambert, Charles Bailly, Alix Goguey, and Laurence Nigay. 2026. *µPoly: a First Microgesture Toolkit*. *Proc. ACM Hum.-Comput. Interact.* 10, 4, Article EICS004 (June 2026), 30 pages. https://doi.org/10.1145/3815367

1 Introduction

The expansion of the field of research on microgesture-based interaction has led to the availability of a multitude of systems for detecting and recognizing rapid and subtle finger movements, known as microgestures. Thanks to these new systems, new interaction techniques based on microgestures have been designed and tested. In particular, microgestures can enable interaction in various contexts, e.g. mixed reality [9] or moving vehicles [16], whether holding an object or not [7, 47]. The growing variety of recognition systems, both in sensing devices and recognition algorithms, makes it more difficult to find and test the best system for a particular set of microgestures in a specific context.

Microgesture recognition systems vary considerably in terms of sensing devices and recognition algorithms. Sensing devices come with different sensors (e.g. force-sensitive sensors [52], IMU sensors [47], cameras [45]) and different form factors (e.g. gloves [18, 52], rings [6, 51], bracelets [14, 43]), while recognition algorithms follow different methods (e.g. threshold-based [18, 52], SVM [6, 53], random forest [23, 24]). Each approach has its benefits and limitations. For instance, some motion tracking systems offer accurate tracking of the finger joints but require multiple cameras and a glove with tracking points [45]. However, this type of detection is neither applicable nor ecologically sustainable in everyday use. Conversely, a bracelet seems more relevant, as it can be integrated into a smartwatch band [4, 14]. Nevertheless, this approach offers a limited number of available sensors, which reduces the number of recognizable microgestures. Alongside the multitude of proposed recognition systems, the current literature on microgestures explores and structures the space of possibilities in terms of microgestures by conducting elicitation studies [5, 11, 46, 50, 55], establishing design spaces [10, 25, 48] as well as designing and evaluating interaction techniques based on microgestures [9, 19, 52, 54]. However, as more and more studies create and evaluate interaction techniques based on microgestures [9, 19, 52, 54], designers and researchers are faced with the need to test different sets of microgestures as well as different sensing devices and/or recognition algorithms before finding the most suitable ones. Despite this need, the absence of standards for microgesture recognition systems as well as the lack of tools for rapid prototyping and exploration are slowing progress in microgesture research [37].

To address this challenge, we propose µPoly, a microgesture recognition toolkit for designers and researchers to lower the cost of entry and exploration of microgesture interaction. µPoly is a toolkit based on state machines that are driven by low-level events. These low-level events are based on

the recently introduced notation for microgestures: µGlyph [10]. µGlyph describes a microgesture by breaking it down into elementary microgestures, i.e. biomechanical movements with properties such as the microgesture actuator. These µGlyph elementary microgestures are used as a standard for low-level microgesture events in µPoly. Compliance with this µGlyph standard for the format of recognition systems events is a central feature of µPoly, enabling easy and seamless testing of different sensing devices and recognition algorithms. This approach is similar to the way different mice produce the same mouse events recognizable by any computer. These low-level events can be combined to describe more complex microgestures. To do so, µPoly relies on µGlyph descriptions implemented as state machines, as presented in Figure 1. State machines enable us to explore large and diverse microgesture sets. They also offer great flexibility, making it easier to modify the recognized set of microgestures. Finally, as state machines describe µGlyph descriptions of microgestures, µPoly can be used together with the µGlyph website, µDex [8]. This allows users to search for a microgesture using text descriptions, images, or articles' metadata on µDex, then simply copy and paste the microgesture µGlyph descriptions into µPoly. As a result, µPoly can be used by experts and novices alike in the µGlyph formalism.

The main contributions of this paper are:

- µPoly, a toolkit for microgesture interface design and rapid prototyping with a measured latency to recognize a microgesture two orders of magnitude faster than the time to perform the microgesture itself,
- seven drivers, covering the diversity of recognition systems in terms of sensors, form factors, and recognition algorithms, as well as five featured applications to demonstrate the usability, flexibility, and capacity of µPoly.

2 Related work

To the best of our knowledge, there is no existing toolkit for microgestures. However, a wide variety of microgesture recognition systems already exist. Our related work first covers this variety of microgesture recognition systems and examines them from three aspects: sensor types, device form factors, and recognition algorithms. We then present how to model inputs, in particular microgestures, using a formal language. Finally, we present guidelines from the literature for designing an effective toolkit.

2.1 Variety of microgesture recognition systems

When designing microgesture recognition systems, three choices need to be made: 1) the type of sensor used, 2) where and how to position these sensors, and 3) the type of recognition algorithm used to process these sensors' outputs. Table 1 presents an overview of the different microgesture recognition systems presented in this related work.

2.1.1 Sensors. A microgesture is composed of a sequence of elementary biomechanical movements, i.e. flexion, extension, abduction, and adduction, performed in contact with a surface or not [10]. Based on this microgesture decomposition, sensors can be divided into two types. ***Component sensors*** can be used to recognize a specific component of a microgesture, e.g. an elementary biomechanical movement and/or a contact information. Such sensors include flex sensors [52] (finger movement), point contact sensors [18, 19, 48] (contact), linear contact sensors[1] [18, 52, 54] (contact + movement in contact) or force sensors [52, 54] (contact + force information). By contrast, ***expression sensors*** can be used to recognize a microgesture without the need to recognize each component element independently. Such sensors include cameras [29, 33, 45, 48], radars [23, 32],

[1]A linear contact sensor can either be a line of point contact sensors [18, 52] or a single sensor [54]

IMU sensors [15, 47], bio-impedance sensors [51], bioacoustic sensors [14, 24] or EMG sensors [42, 43].

2.1.2 Form factors of the sensing device. Sensors have been integrated into sensing devices with various form factors. As in the three approaches to augmented reality [34], these devices augment either the user, a held object, or the environment. In the case of user augmentation, different form factors have been studied. **Gloves –** They make it easy to place sensors on areas of interest, such as phalanxes or specific joints. However, gloves are intrusive and reduce the user's tactile perception by covering the skin surface of the hand. This second aspect can be nuanced by finding a compromise between the surface covered, e.g. by using mittens, and the quantity of sensors that can be attached to the gloves. In research, gloves are often used as ad-hoc prototypes for studies on microgesture interaction [18, 45, 52, 54] because they are easier to build than other solutions. **Exo-skeletons –** They enable sensors to be positioned on the back of the hand, thus limiting the impact of the device on the user's tactile perception. A limited number of sensors can be used with this type of device. For instance, contact and pressure sensors cannot be placed on the back of the hand. In the literature, this type of device has been used in combination with IMUs [15, 47]. **Bracelets –** They provide a lightweight, low-intrusive device. They seem to be the form factor envisioned by Meta [17] and Apple [4] for microgesture recognition. In the literature, they have been used with bio-acoustic sensors [14], EMG sensors [43], radars [23] as well as cameras [53]. **Rings –** They also provide a lightweight, low-intrusive device. In the literature, they have been used with linear contact sensors [6], bio-impedance sensors [51], and cameras [12].

2.1.3 Recognition algorithms. Two main approaches can be identified in the literature: ad-hoc and machine learning algorithms. Using microgesture *component sensors* placed strategically on the hand, it is possible to empirically associate sensor values with µGlyph symbols. Such ad-hoc algorithms have been used with pressure and flex sensors [52] as well as contact sensors [18].

The second approach is to use machine learning algorithms. This approach can also be used with microgesture *component sensors*. For instance, Nailz uses contact sensors on the user's fingernails [31]. These sensors are used to build up an image of the contact between the thumb and the nail. A simple decision tree is then used to classify these images. In the majority of cases, this machine learning approach is used with microgesture *expression sensors* due to the large amount of information collected. Soliman et al. use a convolutional neural network for detection with an RGB-D camera [48]. Saponas et al. first use signal processing to extract characteristics from the signal supplied by EMG sensors then use a support vector machine [42, 43]. SoloFinger initially uses TsFresh [13] to extract features from the signal received from an infrared camera system, followed by a random forest to classify the signals and recognize microgestures.

In both cases, most of the algorithms in the literature recognize microgestures using only the data from the sensors. However, in certain contexts, it is possible to use additional information. For instance, TipText uses a matrix of contact sensors on the tip of the index finger to create a one-phalanx qwerty keyboard [56]. The lack of precision, due to the width of the thumb compared with the size of the keys, means that the user presses several keys on this keyboard at once. As a result, TipText uses information from a matrix of point contact sensors and a statistical decoder to associate the sequence of keys typed with the words in the dictionary to correct potential errors, much like the auto-corrector on our smartphones' keyboards.

2.2 Modeling microgesture interaction

Newman proposed a programming language based on state machines to simplify the development of graphical interactions [39]. Since then, researchers have modeled graphical interaction with

		Boldu et al. [6]	Chan et al. [12]	Deyle et al. [14]	Dobinson et al. [15]	Faisandaz et al. [18]	Faisandaz et al. [19]	Iravantchi et al. [23]	Iravantchi et al. [24]	Lee et al. [31]	Lien et al. [32]	Liu et al. [33]	Saponas et al. [42, 43]	Sharma et al. [45]	Sharma et al. [47]	Soliman et al. [48]	Song et al. [49]	Waghmare et al. [51]	Wambecke et al. [52]	Way et al. [53]	Whitmire et al. [54]	Xu et al. [56]
Sensors																						
Component	Flex sensor																		×			
	Force sensor																		×		×	
	Contact sensor	×				×	×			×						×	×					×
	Linear contact sensor	×				×				×							×		×		×	
Expression	Camera		×									×		×		×				×		
	Radar							×			×											
	IMU				×					×					×							
	Acoustic			×					×													
	Bio-impedance																	×				
	EMG												×									
Form factors																						
User	Glove					×	×												×		×	×
	Exo-squeleton				×					×					×							
	Bracelet			×				×					×							×		
	Ring(s)	×	×															×				
Physical objects																	×					
Environment									×		×	×		×		×						
Recognition algorithms																						
Ad-hoc algorithm						×	×												×		×	
Machine learning	Decision tree									×												
	Random forest		×					×	×		×			×	×							
	k-NN											×										
	Hidden Markov model			×																		
	Naive bayes											×					×					
	Convolutional neural network															×		×				
	Recurrent neural network				×																	
	Statistical Decoder																					×
	SVM	×										×	×							×		

Table 1. Review of recognition systems used in the microgesture literature according to three aspects: types of sensors, types of form factors and types of recognition software.

different kinds of formalisms such as regular expressions [28], state machines [3], or context-free grammars [26]. Such formalisms can then be used as input language for a framework.

Recently, Chaffangeon Caillet et al. introduced the first notation formalizing the description of microgestures: µGlyph [10]. According to the µGlyph underlying model, a microgesture is made of at least 4 parts: a movement, a context, an actuator, and a surface. The *movement* describes the

elementary biomechanical movement of a finger: flexion (▾), extension (▴), abduction (◂), adduction (▸). Additionally, µGlyph also allows the representation of the absence of movement (■). The *context* describes if the movement is done in the air (○○), in contact with a surface (●●), or if it goes on (○●) or off (●○) a surface. The glyphs for the *movement* and the *context* can then be assembled to describe a microgesture at a high level of abstraction. For instance, the microgesture `touch`, which is a finger doing a flexion (▾) from in the air to in contact with a surface (○●), is represented by $\overset{\circ\bullet}{\blacktriangledown}$.

At a lower level of abstraction, µGlyph allows to specify the actuator of an action and the interaction surface. The actuator is either a finger or a finger joint. It is represented as a subscript of the *movement* glyph. For instance, a `touch` performed by the index is represented by ${}_{\mathrm{i}}\overset{\circ\bullet}{\blacktriangledown}$. The interaction surface can either be a part of the hand, such as another finger/phalanx, or an object. It is represented by the contact glyph (●). For instance, a `touch` performed by the index on the thumb is represented by: ${}_{\mathrm{i}}\overset{\circ\bullet}{\blacktriangledown}\left({}_{\mathrm{t}}\bullet\right)$. Additionally, other properties can be described such as pressure or amplitude.

These are the base symbols to describe an elementary microgesture, in other words, one movement by one actuator on one surface. To combine multiple movements, actuators, or surfaces, µGlyph uses three symbols for three types of combination: sequence (;), concurrent (//), and choice (|). For instance, a `tap` performed by the index and middle fingers on the thumb is represented by: ${}_{\mathrm{i}\,//\,\mathrm{m}}\overset{\circ\bullet}{\blacktriangledown}\left({}_{\mathrm{t}}\bullet\right);{}_{\mathrm{i}\,//\,\mathrm{m}}\overset{\bullet\circ}{\blacktriangle}$.

As an interaction designer, when it comes to choosing which recognition system to use, several challenges arise:

- Checking that the sensors can work in the context of use. For instance, two-part contact sensors [18] cannot work with a handheld object without augmenting the object.
- Choosing a sensing device with form factors that suits the context of use. For instance, a glove is acceptable for designing microgestures in the context of motorcycle, but may not be acceptable in the context of a smart home.
- Verifying that the system can recognize the desired set of microgestures. Since each system recognizes microgestures as a whole, rather than as a composition of low-level events, it is difficult to extend the microgesture set without re-training the entire system.

To address these challenges, we propose µPoly, a toolkit that uses elementary microgestures as low-level events fed into state machines to recognize composed microgestures. Before describing µPoly, we review the desired properties of a toolkit as guidelines for the design of µPoly and how these properties have been supported in existing gesture toolkits.

2.3 Designing a toolkit

Numerous papers describe the properties of a successful toolkit [21, 37, 40]. Ledo et al. provide a review of the literature on how toolkits have been evaluated [30]. In this section, we present the properties that a microgesture toolkit needs to have to solve the challenges we identified above.

We identified five important properties for our toolkit. **Reduce development viscosity** – low threshold ($\mathbf{P_{visc}}$) [21, 37, 40]: a microgesture toolkit should make it faster to test new recognition systems and microgesture sets. **Lower skill barriers** ($\mathbf{P_{skill}}$) [21, 37, 40]: a microgesture toolkit should make it easier for novices to implement microgesture interaction, in particular, by encapsulating common microgestures of the literature into a programmable object to ease their use. **High ceiling** ($\mathbf{P_{ceil}}$) [21, 37, 40]: a microgesture toolkit should make it possible to use any microgesture sets and recognition systems, including future ones. **Rely on common infrastructure** ($\mathbf{P_{infra}}$) [21, 40]: a microgesture toolkit should make use of common data structures and programming languages to

ease development. **Enable fast prototyping** ($\mathbf{P_{proto}}$) [21, 37]: a microgesture toolkit should ease the prototyping of recognition systems and microgesture-based applications.

In addition, we will consider the other guidelines from [21, 37]. **Concise API** ($\mathbf{P_{concis}}$) [21]: a microgesture toolkit should use a concise API that promotes how people should think about microgestures. **Encapsulate successful design** ($\mathbf{P_{encap}}$) [21]: a microgesture toolkit should encapsulate successful microgestures to ease their usage. **Coordinate multiple communicating devices** ($\mathbf{P_{coord}}$) [37]: a microgesture toolkit should ease the coordination of multiple recognition systems.

In what follows, we review previous gesture toolkits to examine how they implement these properties.

2.4 Existing gesture toolkits

Yanghee et al. [38] proposed a framework to recognize mid-air gestures. These gestures have multiple properties that need to be recognized simultaneously, such as arm movement, hand orientation, and hand shape. These properties are heterogeneous because some, like hand orientation and shape, can be captured at any moment, while others, like arm movement, require capturing over time. To recognize these heterogeneous properties in parallel, they proposed using coloured Petri-nets, which is a common structure for managing parallel events ($\mathbf{P_{infra}}$). Users can think of a mid-air gestures as a combination of these properties ($\mathbf{P_{concis}}$).

Gesture Kniter is a toolkit to design hand gesture interaction in mixed reality [36]. Gesture Kniter offers a library of primitives ($\mathbf{P_{encap}}$), i.e. elementary hand movements, which can be combined to create a more complex hand gesture recognizer. With this approach, designers do not need to create one recognizer for each gesture they want to use, which would imply collecting data and training one recognizer for each gesture. Instead, they can easily combine primitive recognizers to recognize composed gestures by using visual scripting ($\mathbf{P_{skill}}$). If a primitive is not in the library, users can create a new primitive by demonstrating the primitive to a camera ($\mathbf{P_{ceil}}$). With Gesture Kniter, users can think about hand gestures as a sequence of elementary hand gestures supporting creativity to implement new hand gestures ($\mathbf{P_{concis}}$).

The gesture recognition toolkit (GRT) is a gesture toolkit created for C++ [20]. The toolkit offers the ability to easily test different recognition algorithms for each gesture ($\mathbf{P_{visc}}$). Similar to Gesture Kniter, the GRT is built on a set of core modules that can be linked in pipelines. This facilitates the creation of a complex gesture recognizer by building on top of gesture primitives recognizers ($\mathbf{P_{skill}}$). Finally, the GRT enables common C++ code architecture ($\mathbf{P_{infra}}$).

Proton [28] and Proton++ [27] are frameworks for recognising multi-touch gestures using regular expressions. These regular expressions represent touch events and their properties using a set of predefined symbols and layouts ($\mathbf{P_{concis}}$). To handle parallel events that occur in multi-touch gestures, Proton provides a tablature view of the regular expressions, where vertically aligned events must occur simultaneously. These elements enable the description of any multi-touch gesture ($\mathbf{P_{ceil}}$).

The TouchID toolkit [35] was created to simplify the exploration of new tabletop touch interaction relying on different hand parts/sides and user recognition ($\mathbf{P_{ceil}}$). The toolkit is composed of a visual editor to simplify gesture declaration ($\mathbf{P_{skill}}$) and an API relying on the commonly used event/subscriber model to receive notifications about gestures and hand parts ($\mathbf{P_{infra}}$).

The SoD-Toolkit [44] makes it easy to coordinate multiple sensing devices by integrating a fusion motor in the toolkit ($\mathbf{P_{coord}}$). The same fusion engine enables new devices to be prototyped by integrating data both from existing devices and novel sensors ($\mathbf{P_{proto}}$).

Table 2 summarizes the properties of the presented gesture toolkits/frameworks.

In the following section, we present how µPoly has been designed to support these properties.

Paper	P_{visc}	P_{skill}	P_{ceil}	P_{infra}	P_{proto}	P_{concis}	P_{encap}	P_{coord}
Colored Petri-Net [38]	?	✗	✓	✓	✗	✓	✗	✗
Gesture Kniter [36]	✓	✓	✓	✓	✗	✓	✓	✗
GRT [20]	✓	✓	?	✓	✓	✗	?	?
Proton [27, 28]	?	✗	✓	✓	?	✓	✗	✗
TouchID Toolkit [35]	?	✓	✓	✓	✗	✓	✗	✗
SoD-Toolkit [44]	?	✗	✓	✓	✓	✗	✗	✓

Table 2. Summary of previous gesture toolkits and frameworks, and whether they verify the toolkit properties. ✓, ? and ✗ are derived from our own understanding of the paper since they did not explicitly describe these properties.

3 Underlying Concepts of μPoly

µPoly is a microgesture toolkit that allows easy and fast testing of different recognition systems, both sensing devices and recognition algorithms, as well as microgesture sets. In the specific case of µPoly, the recognition algorithms only recognize elementary microgestures, which are then used as low-level events fed into state machines, as illustrated in Figure 1. In this section, we present the design of µPoly and explain how it operates using low-level events and state machines. In the following section, we detail how to use µPoly.

3.1 µPoly design

Following toolkit design guidelines in the literature [21, 37, 40], the design of µPoly is as follows: **1/ reducing development viscosity (P_{visc})** – µPoly is based on µGlyph, a microgesture notation, which serves as a standard to easily interchange between a wide variety of recognition systems, and connect them to any microgesture-based applications; **2/ high ceiling (P_{ceil})** – µPoly uses µGlyph descriptions of elementary microgestures as input, allowing users to combine them into composed microgestures . In addition, µPoly allows developers to create a new recognition system that can be added to µPoly; **3/ lower skill barriers (P_{skill})** – as µGlyph is an expert notation, µPoly includes a library of commonly used microgestures to simplify their use by novice µGlyph users, as well as a µGlyph visual editor to create ad-hoc microgesture expressions without requiring coding; **4/ rely on common infrastructure (P_{infra})** – µPoly uses an event/subscriber websocket architecture to notify the detection of a microgesture, allowing any third-party applications to integrate microgesture-based interaction; **5/ enable fast prototyping (P_{proto})** – µPoly can also simulate elementary or composed microgestures to test a set of microgestures or an application without the need for a recognition system.

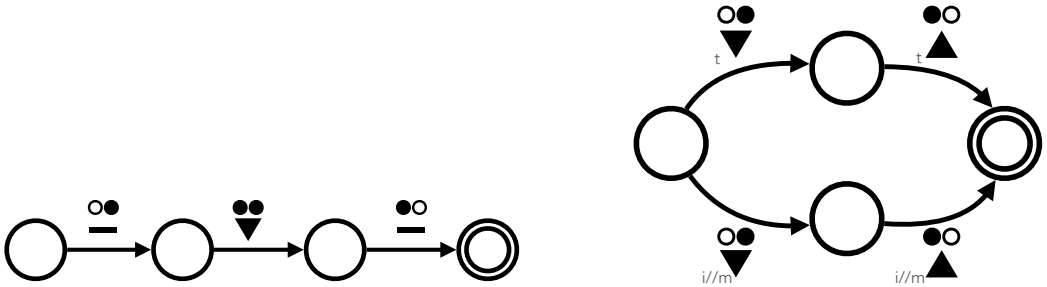

Fig. 2. Examples of microgesture state machines. Left: the state machine represents a `swipe inward`: ; ; . Right: the state machine represents a `tap` of the thumb or a `tap` of the index and middle finger: (;) | (;)

3.2 μPoly model

To detect microgestures, μPoly relies on low-level events, representing elementary microgestures, and state machines, representing composed microgestures.

3.2.1 Low-level events. To introduce low-level events, let's first look at a very common example: mouse clicks. To handle mouse clicks, developers have access to three events of two kinds: two low-level events, `press` and `release`, and one composed event, `click`, which is a sequence of a `press` low-level event followed by a `release` low-level event. With existing microgesture recognition systems, developers only have access to high-level events. In particular, for systems that rely on machine learning, systems have been trained to recognize composed microgestures, but not the elementary microgestures, or low-level events, from which they are composed. However, as explained in GestureKnitter [36], training recognizers for elementary microgestures and their different characteristics, e.g. the body part that triggered the microgesture or the amount of pressure applied during the microgesture, offers greater flexibility by enabling elementary microgesture recognizers to be combined in order to recognize composed microgestures.

This approach is particularly interesting for microgestures. Many microgestures share the same elementary microgestures, as shown in the appendix of the μGlyph article [10]. For instance, we consider a set of three common microgestures: `tap` (▼; ▲), `swipe inward` (▼; ▼; ▲), and `swipe outward` (▼; ▲; ▲). These microgestures are composed of 4 elementary microgestures: ▼, ▲, ▼, ▲. These 4 elementary microgestures can be reassembled to recognize at least 10 different microgestures: `touch` (▼) [11], `release` (▲) [55], `tap` (▼; ▲) [45], `drag inward` (▼) [45], `drag forward` (▲) [45], `swipe inward` (▼; ▼; ▲) [52], `swipe forward` (▼; ▲; ▲) [52], `drag&drop inward` (▼; ▲) [55], `drag&drop forward` (▲; ▲) [55], and `ream` (▲; ▼) [55].

3.2.2 State machines. State machine approaches are particularly well suited for microgesture interactions, which involve small-amplitude, rapid, and sometimes subtle movements. Whereas body gestures in general can involve a very large set of physical movements of varying amplitudes (which makes body gestures difficult to break down into axioms), microgestures, as explained in *μ*Glyph [10], rely on only a small, fixed number of biomechanical actions (finger flexion ▾ , extension ▴ , abduction ◂ , and adduction ▸) along with characteristics (e.g., index, in contact). The low-level vocabulary to be recognized for microgestures is considerably reduced compared to body gestures. This is why we draw a parallel between the management of microgestures and that of mouse events. In fact, if we further develop this parallel already introduced in the previous section, mouse `press` and `release` are the *biomechanical actions*, and left and right buttons are their *characteristics*. Therefore, in the same way as for the management of mouse and touch events [3, 27, 28], we use state machines with low-level events, i.e. an elementary microgesture consisting of a biomechanical action coupled with characteristics. State machines will thus describe sequences of elementary microgestures, i.e. a composed microgesture, and since we rely on μGlyph to describe elementary microgestures, each state machine can be represented by the μGlyph description of the composed microgesture. μGlyph descriptions thus act as regular expressions describing composed microgestures. Figure 2 presents two examples of state machines and their associated μGlyph descriptions.

Overall, μPoly uses elementary microgestures as input, making it easy to switch from one recognition system to another ($\mathbf{P_{visc}}$) and to add or modify on the fly the set of recognized microgestures by simply adding or tweaking the corresponding μGlyph expression ($\mathbf{P_{ceil}}$).

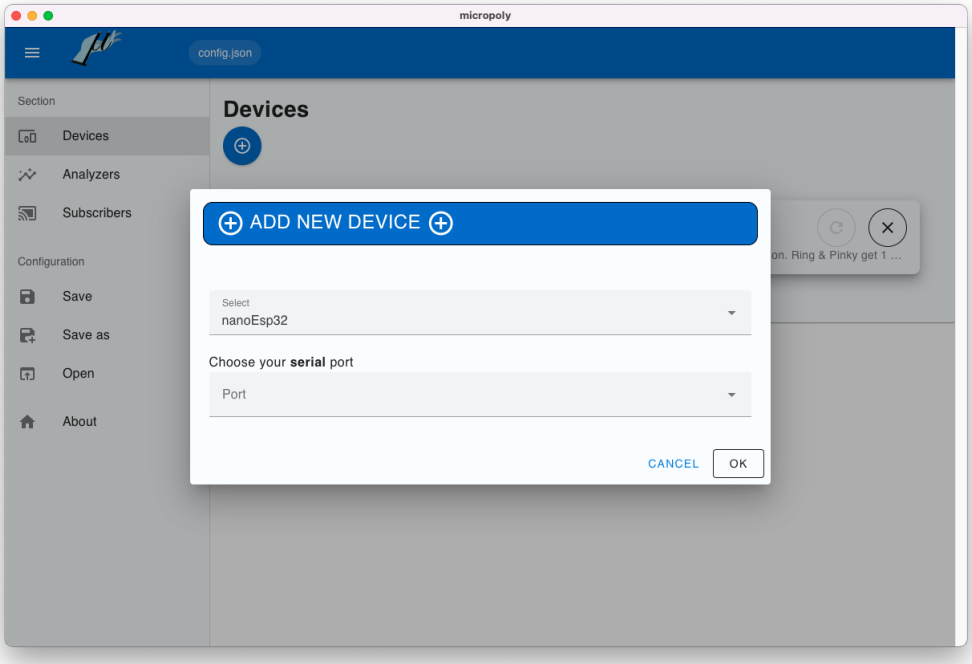

Fig. 3. µPoly panel to select a recognition system

4 µPoly description

µPoly is divided into two parts: a core and a graphical user interface (GUI). The core of µPoly is developed using Node.js. It is responsible for communicating with sensing devices and applications as well as managing state machines. The user interface is developed using Vue.js. The user interface is used to simplify core configuration by providing an interface for selecting a recognition system, specifying the microgestures to be recognized and connecting to an application. µPoly can run without the user interface if required. The core and user interface can be compiled using Electron into an application that can be used on a computer or smartphone. All the source code, binaries and documentation are open-source and available on the project repository webpage[2].

In the following, we present how developers and practitioners can configure and use µPoly. We divide the description into 3 topics: setting up an existing recognition system or a new one (parts A and B of Figure 1), specifying microgesture recognizers (part C of Figure 1), and setting up a third-party client application (parts D and E of Figure 1).

4.1 Selecting a new recognition system

One of µPoly's main objectives is to make it easy to switch from one recognition system to another. To use a recognition system, a user first connects the sensing device to the computer on which µPoly is installed. The connection type depends on the device communication protocol (e.g. USB cable for serial port). Once the device is connected, it can then be configured in the system: users must select from a drop-down menu of existing drivers the corresponding one, and setup the

[2]removed-for-submission, see the supplementary material

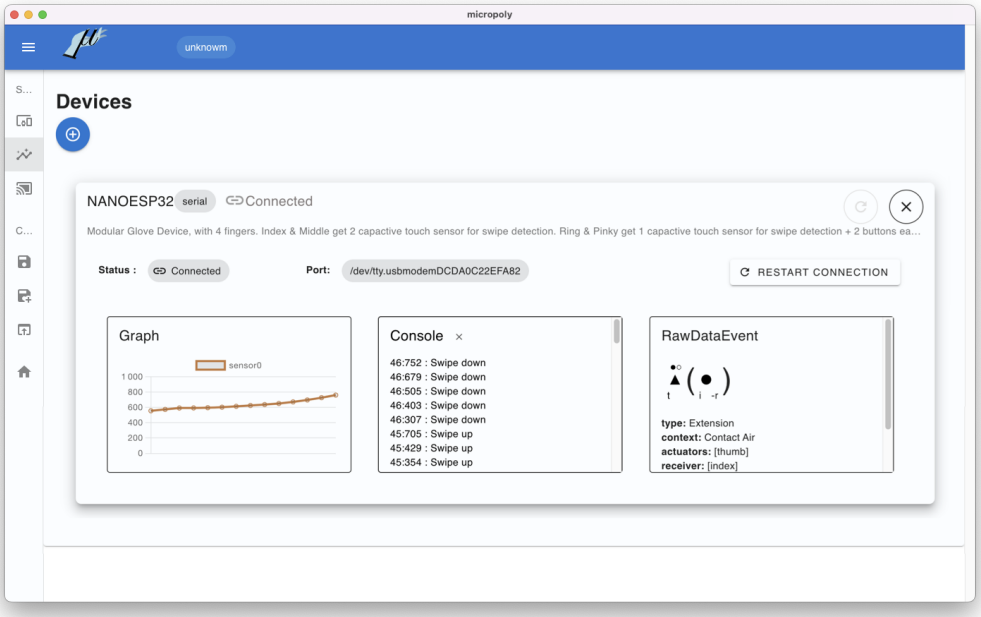

Fig. 4. µPoly panel to debug a recognition system

connection properties (e.g. which serial port), see Figure 3. Drivers include the communication protocol and recognition algorithms for transforming raw data from sensing devices into low-level events, i.e. elementary microgestures. Different drivers with different recognition algorithms can be implemented for the same sensing device. Therefore, testing different recognition algorithms for microgestures is equivalent to testing different drivers. In addition, several recognition systems (possibly of the same type) can be active at the same time. As a result, µPoly can also be used as a tool for coordinating multiple recognition systems ($\mathbf{P_{coord}}$).

4.2 Developing a new recognition system

To integrate a new recognition system into µPoly, developers need to first implement a driver for their sensing device. It is a Javascript object class integrated into µPoly. Each driver transforms the raw data stream transmitted by the device into a stream of elementary microgestures that can then be processed through the state machine pipeline of µPoly. In their driver, the developers specify the communication protocol used between the device and µPoly. **Reusing known architectures** ($\mathbf{P_{infra}}$), µPoly offers methods to simplify the use of common communication protocols [9, 52], i.e. serial port, websockets and BLE. Methods for other communication protocols will be added, but developers can already use other communication protocols by manually adding the appropriate libraries. To help implement the driver, we provide one generic skeleton class and one skeleton class for each communication protocol (see documentation[3]). We also provide 7 drivers for 7 existing sensing devices, including two from published papers [9, 18], one for the HoloLens 2 hand tracking system [1], one for Google Mediapipe [2], and three for gloves from unpublished papers. Some of these drivers are presented in the application section. To facilitate driver debugging, µPoly

[3]see Appendix A and the supplementary material

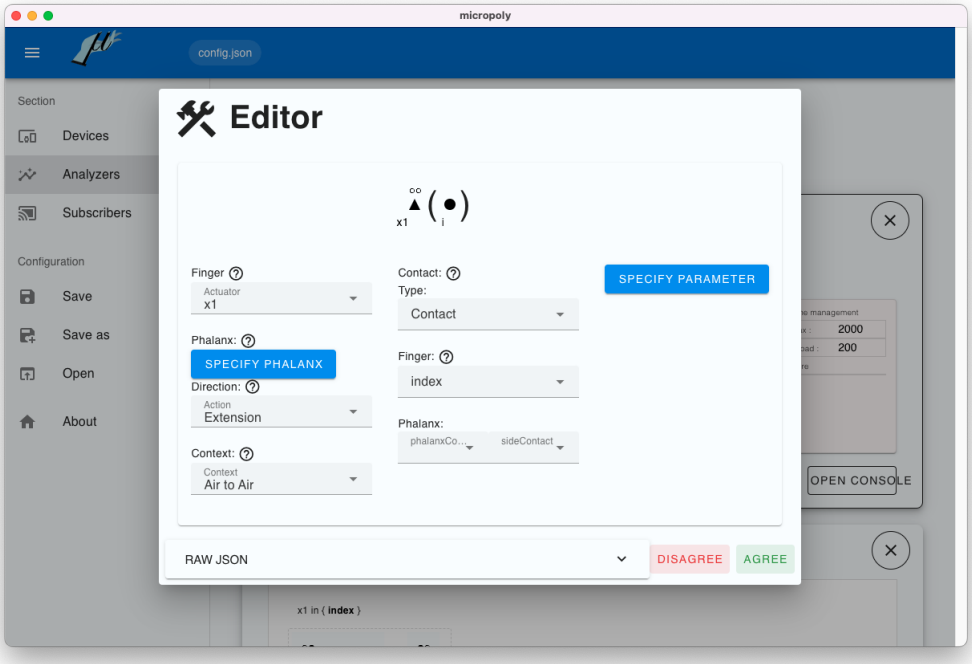

Fig. 5. Visual editor to input µGlyph descriptions.

offers methods for printing and graphically displaying the real-time raw values transmitted by the sensing device, see Figure 4 ($\mathbf{P_{proto}}$). µPoly also displays the elementary microgestures outputted as low-level events. The structure of these events is inspired by the µGlyph data structure from µDex [8], an online database of 100+ microgestures. However, we use a simpler and more concise structure since we only need to focus on low-level events ($\mathbf{P_{concis}}$). Examples of the structure are shown in the documentation[3] of µPoly.

Ultimately, any sensor and recognizer used to recognize microgestures can be used with µPoly simply by adapting the format of the output produced by the recognizer. More details are available on the project repository webpage[2].

4.3 Declaring a microgesture to recognize

µPoly uses the µGlyph notation to define the microgestures to be recognized. To specify a microgesture recognizer, µPoly embeds a visual editor of µGlyph expressions, shown in Figure 5. This visual editor allows users to enter any expression that complies with the µGlyph notation.

Since the µGlyph notation is intended for expert users, µPoly integrates two features, shown in Figure 6, that aim to **reduce the barrier to entry**. First, µPoly offers a library of common microgestures described by name and image that can be used without the need to write the corresponding µGlyph expressions ($\mathbf{P_{encap}}$). Second, µPoly uses the same data structure to represent a µGlyph description as µDex [8] does. It gives users access to a database of over 100 microgestures extracted from the literature that can directly be imported into µPoly simply by copy-pasting µGlyph descriptions. It also allows users to crawl through these microgestures using text descriptions,

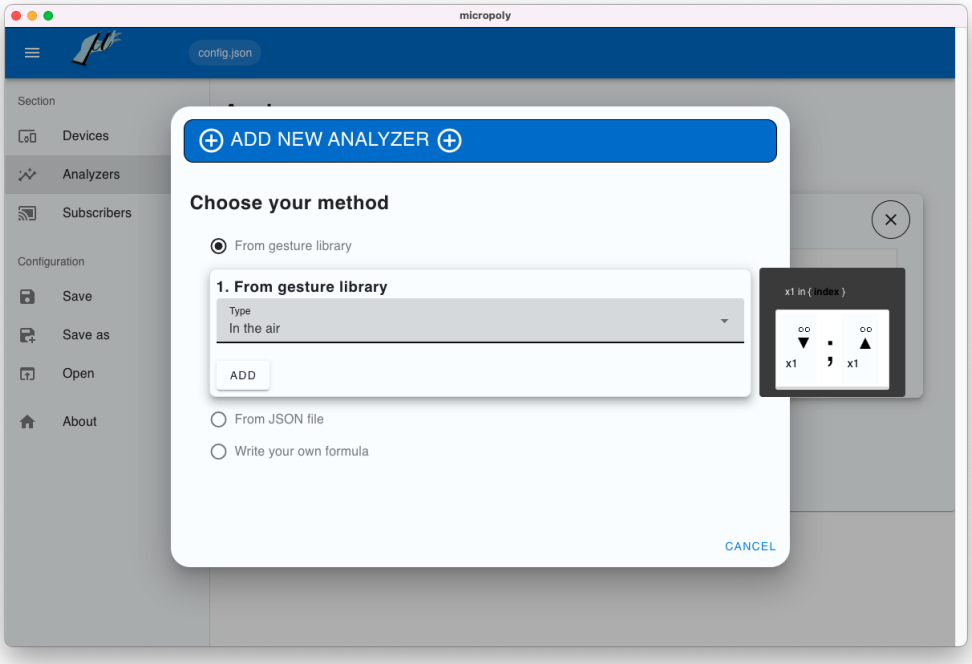

Fig. 6. To ease the use of μPoly, users can choose existing microgestures from an integrated library or from the μDex

image descriptions, authors, papers, queries, and more. So, even though μPoly is based on an expert notation, users are not required to know it ($\mathbf{P_{skill}}$). By contrast, users familiar with μGlyph can use the visual editor to add microgestures, including those that have never been used before ($\mathbf{P_{ceil}}$).

Finally, μGlyph expressions are used to visually represent the internal state machines. With each new elementary μGlyph microgesture (i.e. low-level event) supplied by the devices, visual feedback informs users of the current active state of the state machine, see Figure 7. To ease software development and avoid being bound to the implementation timeline of a device, μPoly can simulate any low-level events to test state machines coverage and transitions, for instance to identify potential conflicts between state machines (see documentation[3]). Moreover, multiple state machines are usable in parallel if needed. Nonetheless, a careful microgesture mapping design is then required to avoid triggering multiple microgesture events simultaneously. This approach follows typical behaviour from touch and mouse events where a click also triggers a press and a release event.

4.4 Subscribing and receiving a microgesture

Once devices and microgestures have been configured, users can transmit recognized microgestures to a third-party application via a subscription mechanism using websockets and JSON format. Websockets and JSON can easily be implemented in a wide range of programming languages[4]($\mathbf{P_{infra}}$). In particular, multiple applications can be connected at the same time. Within this architecture,

[4]js, python, java, c#, and more: https://en.wikipedia.org/wiki/WebSocket and https://en.wikipedia.org/wiki/JSON

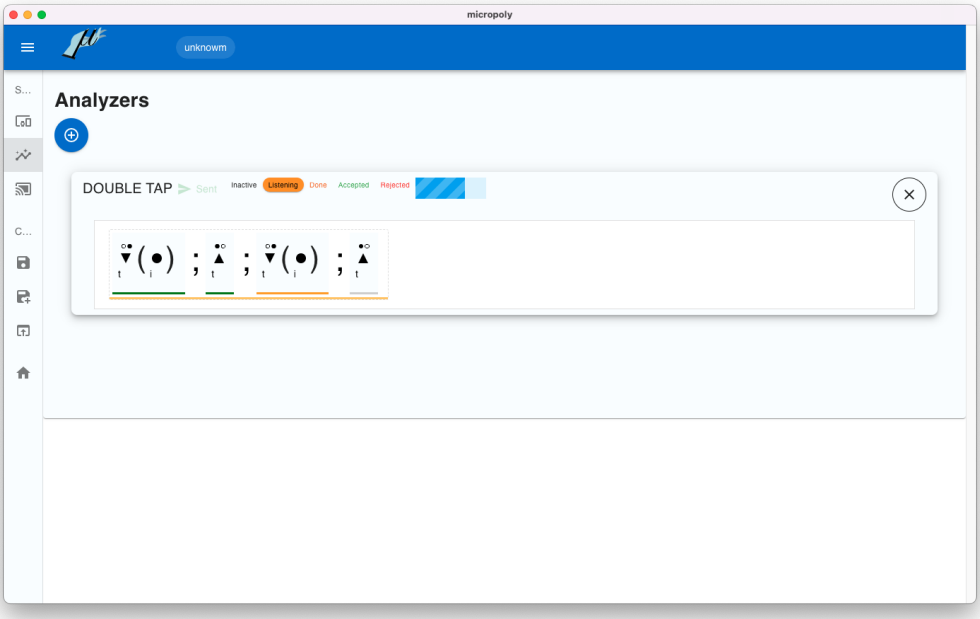

Fig. 7. µPoly provides visual feedback on the current state of each state machine: green lines represent successful transitions and orange lines potential transitions. A line below the full µGlyph expression represents whether the microgesture is waiting for potential transitions (orange line) or whether it is recognized (green line).

µPoly acts as a websocket server and third-party applications as websocket clients. The websocket server has been implemented using socket.io[5]. To subscribe to a microgesture event, the users must first declare their application in the µPoly interface, see Figure 8, as well as the port on which the application is to listen. Once the application has been declared, the users select the microgesture events to which the application is subscribed. Each time a microgesture in the set is recognized, the corresponding information is sent in JSON format to the application. To facilitate application development and avoid being tied to a device's implementation schedule, µPoly can simulate the output of state machines, enabling microgesture events to be sent manually to subscribed applications (see documentation[3]).

5 *µ*Poly Scenarios and Applications

µPoly can be used to develop any application supporting microgesture-based interaction. Application examples include interaction in a car [16], in a cockpit [52], and in mixed reality [9, 54]. In this section, we describe two scenarios: a menu control in augmented reality, and a video player control. At the end of this section, we present applications, which we implemented and provide, that can be used without additional installation to get started with µPoly.

[5]https://socket.io

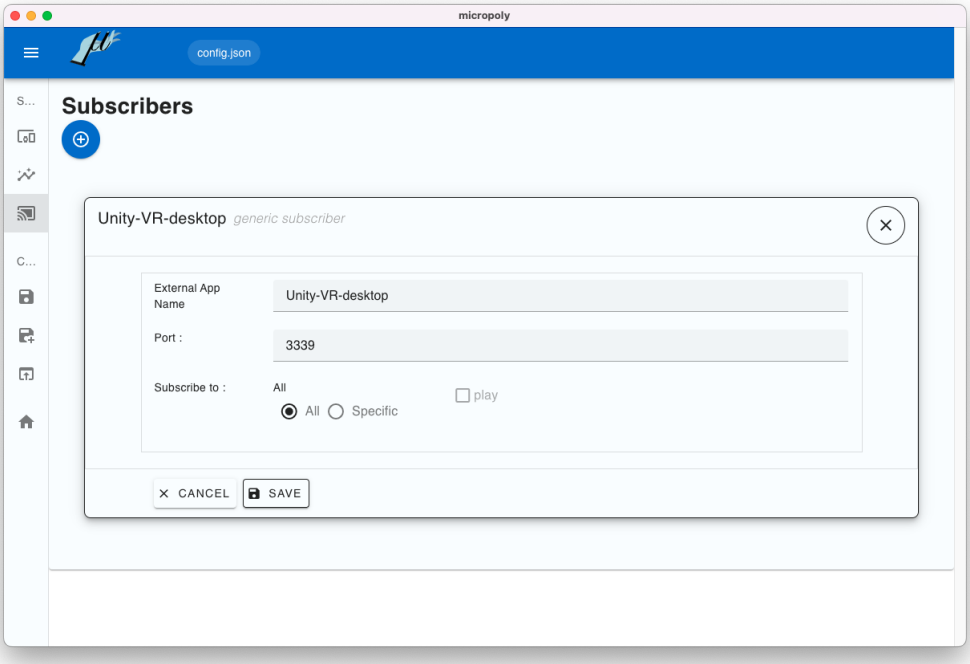

Fig. 8. µPoly panel to create a client application. Microgestures can be simulated using the bottom panel.

5.1 Scenario 1: Developing an AR application using μPoly

The application described in this scenario is inspired by the future work section of Chaffangeon Caillet et al. [9]. The various steps involved in developing this application illustrate the properties expected of a toolkit, as described in the *Designing a toolkit* section of the related work. The accompanying video figure demonstrates these steps.

Prototyping the application – Vic is a software developer of AR applications for HoloLens 2. She recently discovered that microgestures are a promising new modality for AR. Vic would like to use microgestures to control a menu in her application. She comes across µPoly which can use the HoloLens 2's camera to recognize microgestures. First, she creates a prototype of her application in Unity. Then, Vic connects her Unity application to µPoly via a websocket, reusing websocket snippets she used in previous research projects [9, 41] ($\mathbf{P_{infra}}$). Vic now needs to decide which microgestures she wants to use. She browses the µDex catalogue, and settles on potential candidates. Not knowing µGlyph, Vic copy-pastes the corresponding expression directly from µDex to µPoly. As she is developing her application on a computer, she wants to carry out some quick tests using her computer's camera. So she selects Google MediaPipe from the list of µPoly pre-packaged drivers and performs the microgesture candidates in front of her computer's camera.

Modifying the microgesture set – Having explored and reused several µDex descriptions of microgestures, Vic now wants to specify her own microgestures ($\mathbf{P_{ceil}}$). Using µGlyph, she understands that microgestures are a sequence of low-level elementary microgestures ($\mathbf{P_{concis}}$). She therefore sets out to integrate visual and audio feedback on the microgestures intermediate steps. She first focuses on her `tap` microgesture (☟;☝) that clicks a button. She wants the button to light

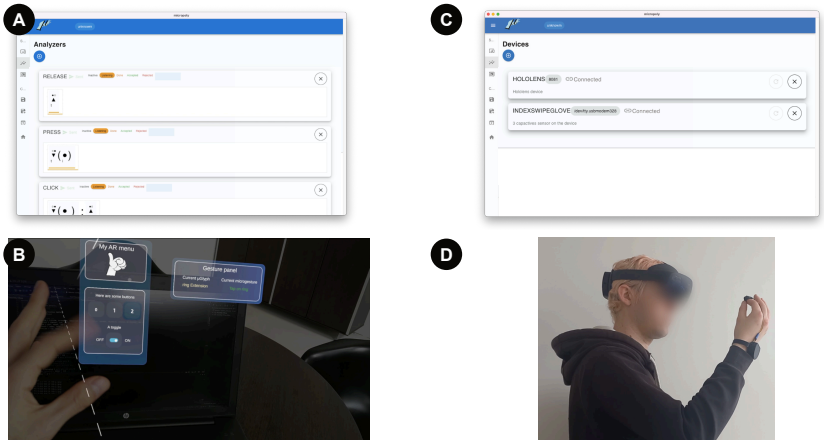

Fig. 9. Scenario 1: Developing an AR application using µPoly. A) Vic defines a click and its two component microgestures: press and release. The state machines associated with these microgestures are run through in parallel. B) In the AR application, a click triggers a button. The press and release microgestures provide visual feedback, indicating that the button is pressed and then released during the click. C) Vic defines two devices: the HoloLens 2 built-in camera with Mediapipe and a custom-built ring. D) These two devices are used in parallel by the user.

up after the `press` (🤏), and to emit a sound after the `release` (✋). To detect these microgesture intermediate steps, she creates two new recognizers in µPoly by duplicating twice the µGlyph recognizer of a `tap` (Figure 9A). In both cases, she trims down the expressions to keep only the elementary microgesture needed (`press` or `release`). However, she does not need to modify the recognition system nor the µPoly application code. She then adds both recognizers to her HoloLens application µPoly websocket subscription ($\mathbf{P_{visc}}$).

Deploying on HoloLens – Now that Vic has her definitive set of microgestures, she goes on to deploy her application on HoloLens. She first adds the HoloLens 2 camera pre-packaged driver ($\mathbf{P_{visc}}$) and specifies the websocket through which connecting the Hololens 2 Camera (Figure 9b). She then asks Charlie, a beta-tester, to try out the application (Figure 9c). Charlie notes that to ensure the HoloLens detects her microgestures robustly, she has to keep her hands in front of the camera. After a few minutes, she begins to fatigue because of holding her arm up (the so-called "gorilla arm" effect [22]). Vic concludes that the HoloLens camera is not the best option for detecting microgestures for a prolonged period of time. She'd rather have her users have a relaxed posture, such as arms alongside the body. She needs to explore another approach.

Testing and prototyping different recognition systems – Aure, a hardware developer on Vic's team, suggests testing two new form factors: a glove, inspired by [52], and a ring, inspired by [6]. After prototyping both devices, Aure's next step is to develop two drivers for µPoly. Aure reads the documentation, takes a look at the other pre-packaged-drivers source code using serial connection, and goes on to fork the driver example with the already implemented serial connection. She uses µPoly visualisation of the devices raw data to implement and debug her driver, ensuring her devices' driver correct integration to µPoly ($\mathbf{P_{proto}}$). After trying out both devices ((Figure 9d)), she settles on the ring being less intrusive, even if it cannot recognize all the microgestures Vic envisioned for her application.

Using two recognition systems – Vic and Aure talked it through and found a suitable trade-off: using both the ring and the HoloLens camera to detect the microgestures ($\mathbf{P_{coord}}$) (Figure 9C).

The ring, albeit recognizing a subset (i.e., `taps` and not `swipes`), can be used in a relaxed position and recognizes the most commonly used microgestures of the application. While occasional microgestures (e.g., `swipes`) can be robustly recognized by the HoloLens camera.

5.2 Scenario 2: Implementing a continuous control with μPoly

The application described in this scenario illustrates a more complex use of the toolkit. More information about the implementation of this application can be found in the µPoly's documentation [6].

Discrete control – Grace is a developer for a video streaming company. She was tasked to map out some of the video player controls with inputs that would be enabled with the next generation of wearable devices. She found out that microgestures, and especially swipes, are credible candidates. She thus setups a demo prototype of her video player with a glove of the literature to test out mappings ($\mathbf{P_{proto}}$). After having tested her pipeline from glove to video player, she sets up a `swipe outward` (🖐;🖐;🖐) recognizer and a `swipe inward` (🖐;🖐;🖐) one in µPoly. She connects both microgestures to her navigation function. Receiving N `swipes outward` (resp. `swipes inward`) each spaced by less than 200ms, fast forwards (resp. rewinds) 5s ($N = 1$), 10s ($N = 2$), 30s ($N = 3$), 1min ($N = 4$), ...

Continuous control – While satisfied with her results, she would still like to avoid both physical repetition and timeout in her state machines so that the command is sent directly after one swipe is being performed. She therefore decides to break up the swipes as one would do with a mouse drag interaction. She deactivates both her `swipe outward` and `swipe inward` recognizers in µPoly, and sets up four new ones: a `press` (🖐), a `slide outward` (🖐), a `slide inward` (🖐), and a `release` (🖐) ($\mathbf{P_{visc}}$). She then connects all four microgestures in her application. When recognizing a `press`, she counts up the number of `slides outward` and `slides inward` she receives, continuously updating a visual feedforward accordingly ("Jumping X seconds ahead/back?"), and triggers the jump on the `release`. Not only does it require just one swipe, but it also allows to finely control on the fly (in both direction) the value of the jump.

Combining controls – Grace prefers the second option. However, the first one, in her opinion, would not be so bad for volume control. As opposed to her navigation needs, volume can be controlled linearly. With the first option, each swipe could directly increase or decrease the volume by 10%, and no timeouts would be necessary. She therefore re-activates both `swipe outward` and `swipe inward` ($\mathbf{P_{visc}}$), connects them up to her volume controller, and tweaks all recognizers so that volume is now controlled with swipes on the index and navigation with swipes on the middle finger ($\mathbf{P_{proto}}$).

5.3 Existing applications

In addition to the applications used for the user tests (e.g., music player, in the evaluation section hereafter, we developed two others: a keyboard binder and a VLC controller. To enable out of the box prototyping with µPoly, both applications are directly integrated into µPoly[7] The first application allows the user to associate microgestures with keyboard keys. It is therefore possible to control applications such as Keynote or PowerPoint by triggering keyboard shortcuts through microgestures. The second application, developed with the VLC API, lets users control VLC by performing microgestures. This application requires VLC to be installed and its *Lua*[8] interface to be configured.

[6] https://velvet-noise.notion.site/Continuous-Control-2f69cf7618bd80958751c0ba32b5c6f1

[7] We also provide a standalone python code version of both application as examples of third-party application development. Code can be found in the µPoly repository: removed-for-submission, see supplementary material as well as Appendix A.

[8] https://wiki.videolan.org/Documentation:Modules/lua/

Ultimately, thanks to these two integrated applications and the pre-packaged MediaPipe driver, µPoly can be easily be tested out of the box without developing recognition systems or applications.

6 μPoly evaluation

To evaluate µPoly, we performed multiple evaluations to cover all four types of evaluation strategies for a toolkit from the literature, summarized by Ledo et al. [30]. We already presented two types of evaluation:

- **type 1** (i.e., demonstration, individual instances and beyond description) by describing application scenarios and existing applications in the previous section;
- **type 4** (i.e., heuristic) by describing how toolkits properties from the literature (i.e., $\mathbf{P_{visc}}$, $\mathbf{P_{skill}}$, $\mathbf{P_{ceil}}$, $\mathbf{P_{infra}}$, $\mathbf{P_{proto}}$, $\mathbf{P_{concis}}$, $\mathbf{P_{encap}}$ and $\mathbf{P_{coord}}$) were baked into µPoly design.

In this section, we present two additional evaluations: **type 2** (i.e., usage, early usage studies and feedback) and **type 3** (i.e., performance). Table 3 summarizes our evaluations of µPoly.

6.1 Early usage studies (type 2)

We conducted three early usage studies for each part of µPoly, testing all three aspects of the µPoly toolkit (device, recognizer, usage). The aim of these studies was to detect any obstacles or problems in using µPoly for HCI practitioners.

Developing a new recognition system. We carried out an initial test to ensure that it was possible to create a driver for µPoly, using the available documentation. We approached two microgesture researchers (one academic and one industrial) who had each developed a microgesture recognition system. They were already familiar with µGlyph but not with µPoly. We asked them to adapt their already-developed recognition system and integrate it into the toolkit. Both succeeded in creating a driver for the HoloLens 2 and Mediapipe respectively. While building their initial recognition system took about a week, creating a driver for µPoly took about 2-3 days. Both experiences and debriefings taught us that writing a driver for a new device directly for µPoly would not necessarily add complexity to the bulk of the task, and that translating an existing driver into the µPoly ecosystem is very much within reach. Additionally, their comments helped us to fix a few minor bugs in µPoly's GUI for devices, and to clarify parts of the documentation on how to build a driver. Both researchers showed interest in µPoly and were subsequently invited to become co-authors of this article as we proposed to integrate their works as pre-packaged drivers in µPoly.

Declaring a microgesture to recognize. We carried out an initial test to ensure that a user could declare microgestures for recognition, without any prior knowledge of µGlyph notation. We recruited a PhD student who had never worked on microgestures before. We began by introducing her to the concept of microgestures and an overview of µPoly. We then used the official µGlyph tutorial, which lasts around 15 minutes. The aim of the test was to get the participant to write a `tap` and a `swipe`, two of the most common microgestures in the literature, as well as to use microgesture features, such as a specific finger. We set up a glove capable of detecting these two microgestures on the index and middle fingers and its corresponding driver. µPoly has also been configured to communicate with an external custom music player app (see Figure 10). This music player is a simple HTML/JavaScript website[9], which loads up free-to-use songs from a local directory. We provided a simple API to control the music app (e.g. an event[10] named `toggle_play` would control the `play`/`pause` functions). We did not explain the visual microgesture editor, to let the participant

[9]This application can be found, with other examples, in the µPoly repository.

[10]The available events were: `toggle_play`, `play`, `pause`, `stop`, `next`, `prev`, `mute`, `unmute`, `toggle_mute`, `volume_up_X` (to increase volume by X%), `volume_down_X` (to decrease volume by X%), `volume_X` (to set the volume to X%), `song_fwd_X` (to

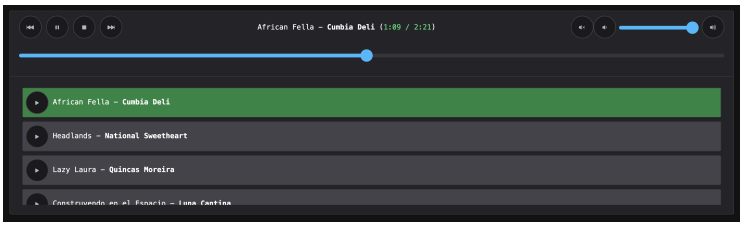

Fig. 10. Custom music player app used in our test to declare microgestures.

discover it by trial and error, and test her expressions live with the glove. Overall, the participant managed to write all instructed microgestures (i.e., `tap` and `swipe` on all fingers but the thumb) in 1h30 (including the introduction and the 15-minute tutorial). We asked the participant to think aloud during the session. The main difficulties were related to the use of variables with μGlyph, but all were overcome by the participant alone. It should be noted that, although available, the participant did not use the documentation during the experiment. Based on our observations and the participant's comments, we have fixed a few bugs in the μGlyph editor included in μPoly.

Subscribing and receiving a microgesture. We carried out two initial tests to ensure that it was possible to link μPoly to an existing application that followed a specific event standard. For the first test, we contacted an engineer at a local living lab. The living lab focuses on home automation and runs a platform integrating a fully functional apartment with smart appliances. The appliances are controlled using the living lab's customized API that follows the openHAB standard[11]. The aim of this test was to connect μPoly to this API. The recognition system and the microgesture recognizers were already set up in μPoly. The participant had to create an application linking the μPoly outputs to the apartment's API. The participant studied the *Subscribers* page and created a python server to control a light and electric blinds in an hour and a half. Apart from retrieving hello-world examples in python and adapting them for the exercise in hand, no particular problems were reported by the participant.

For the second test, we contacted a master student working on collaborative robotics. As part of his internship, the student needed to connect a microgesture recognizer to a ROS node[12] to control a robot. We provided him with a glove for which the recognizer had already been implemented, as well as μPoly documentation. In two full-time days, the master student managed to connect the glove, describe the microgestures and implement a ROS node that can connect to μPoly using a websocket. The master student reported no problems.

6.2 μPoly performance: measuring microgesture detection latency (type 3)

We carried out initial tests to verify the latency introduced by μPoly and its decomposition of microgestures into elementary μGlyph events. Two tests were carried out: one with μPoly and an application running on the same laptop, the other with μPoly and an application running on different computers connected via a WLAN.

6.2.1 Latency on a single computer. For the first test, we used a tactile switch[13] fitted on a glove and controlled by an Arduino micro. The glove is connected to μPoly using a USB cable and the serial

fast-forward X second in the song), `song_bck_X` (to rewind X second in the song) and `song_move_X` (to set the song cursor to X%).

[11] openhab.org

[12] https://wiki.ros.org

[13] Alps Alpine ref. SKUAAAE010

port. The glove emits the event ⚬▼• , when the button is pressed, and the event •▲⚬ , when the button is released. In µPoly, we use a single state machine to recognize a `tap` (⚬▼•;•▲⚬). When recognized, the `tap` microgesture event is transmitted to our custom-built music application using websockets. The application runs on the same computer as µPoly.

Using this setup, we performed 5 `tap` microgestures on the glove button, each separated by several seconds. Each `tap` activates the unmute/mute button of the music application. Using an iPhone 8 in slow-motion mode (240 frames per second), we filmed both the glove, on which `tap` microgestures were performed using a small rod, as well as the screen with the music application running and visible. We also logged the laptop UNIX timestamps when µPoly received serial port data and when µPoly sent websocket messages. First, we made sure that the video was recorded at 240 frames per second[14]. Using the number of frames, we then timed for each `tap`: (1) the duration between the first frame in which the button has been physically down and the first frame in which the button has been physically up (t_{tap}); (2) the duration between the first frame in which the button has been physically up and the first frame in which the screen indicates a change in volume (t_{event}). Using the UNIX timestamps, we have calculated the time between when the last serial data is received (i.e., button up events) and when the websocket message event is sent (t_{proc}). The average t_{tap} – which corresponds to the time required to physically perform a `tap` at a normal pace – is 310 ms (std = 121 ms). The average t_{event} – which corresponds to the time, including network communication, required to process the end of a movement to trigger a command – is 42 ms (std = 9 ms). The average t_{proc} – which corresponds to the time required for µPoly to process and send the data – is 0.8 ms (std = 0.4 ms). Overall, µPoly latency is an order of magnitude less than network latency in the best case (i.e. when network latency is minimized), which in turn is an order of magnitude less than the time required to perform an elementary microgesture.

6.2.2 Latency on multiple computers. For the second test, we compared µPoly's latency with a commercial solution and in a more realistic configuration, i.e. µPoly and the application are not running on the same computer. We used a HoloLens 2 headset which uses `press` microgestures natively (⚬▼•(t•)) as its main gestural interaction technique. To communicate with µPoly, the HoloLens used websockets over WiFi to send and receive events. A participant wore the headset and performed a thumb-index `press` when a virtual circle displayed in front of her turned green. To ensure that the participant could not anticipate when to perform the microgesture, the circle repeatedly turned to another color every 2 to 4 seconds with a 0.5 probability of being green. A total of 24 trials were carried out. We recorded the HoloLens timestamp when the circle turned green and the HoloLens timestamp when a HoloLens `press` microgesture event was received by the application, as well as the HoloLens timestamp when a µPoly `press` event was received by the application. We recorded both conditions at the same time to avoid any bias (e.g., CPU load) in stimulus responses. Performing a `press` took an average of 596.5 ms (std = 83.4 ms) with µPoly and an average of 516 ms (std = 86.2 ms) with HoloLens recognition.

Considering that there is no latency for the HoloLens to recognize the `press` microgesture, the 516 ms would represent the reaction time, to the light turning green, and physical movement to perform the `press` microgesture. Thus, the 80.5 ms difference would represent the network latency, i.e. back and forth communication via WiFi between HoloLens and µPoly, since the previous experiment exposed a µPoly processing time of about 1 ms.

6.2.3 Overall results. In summary, when recognizing a microgesture, the main latency of the µPoly pipeline comes from network handling of inputs and outputs. However, this latency still appears to be an order of magnitude lower than the time required to execute an elementary microgesture.

[14]We display a stopwatch on screen, counting the number of frames in one-second interval, at 3 random points of the video.

Consequently, the latency introduced by µPoly appears to be non-additive since in a sequence of elementary microgestures $(e_1; \ldots; e_n)$, µPoly will finish analyzing each elementary event e_i before one physically finishes executing the next event e_{i+1}. The overall latency, independent of the microgesture length, only depends on the communication latency.

7 Discussion, Limitations and Future Work

We have demonstrated that µPoly can be used with a variety of recognition systems with different sensors (contact sensors, force-sensors, and cameras), different device form factors (glove, ring, augmented environment), and different recognition algorithms (ad-hoc, machine learning). We have also shown how µPoly can be used by two kinds of developers to create microgesture-based interactions: hardware developers and software developers. In this section, we present the achieved and future steps for µPoly.

7.1 Integrating missing µGlyph properties

µGlyph is a notation containing a variety of symbols to describe a microgesture at several levels of abstraction. µGlyph also includes advanced techniques for reducing description size by factoring or encapsulating certain parts of the description. µPoly does not currently incorporate all the properties of the µGlyph notation and therefore does not fully support the expressive power of µGlyph for the implementation of microgesture-based interaction. In this first version, µPoly does integrate all finger, movement, and composition symbols. However, µPoly has some limitations on how composition symbols can be used. OR (resp. AND) composition can only be added within a formula or between movement glyphs. Thus, the OR composition formula ${}_{\mathrm{i\,|\,m}}\overset{\circ}{\bullet}\!\!\blacktriangledown$ needs to be written as $x \in \{i|m\}$, ${}_{\mathrm{x}}\overset{\circ}{\bullet}\!\!\blacktriangledown$. This has no impact on µGlyph's expressive power, as the authors of µGlyph already recommend introducing such variables to avoid potential ambiguities. Similarly, the AND composition ${}_{\mathrm{i\,//\,m}}\overset{\circ}{\bullet}\!\!\blacktriangledown$ needs to be written as ${}_{\mathrm{i}}\overset{\circ}{\bullet}\!\!\blacktriangledown\!\!/\!/{}_{\mathrm{m}}\overset{\circ}{\bullet}\!\!\blacktriangledown$. This has no impact on µGlyph's expressive power, since the use of AND composition between finger symbols has been introduced as an advanced technique to factorize µGlyph descriptions.

µGlyph can be used to describe parts of the hand other than the fingers, such as the phalanx or the finger joint. µPoly can recognize the different phalanxes and finger sides. However, recognition of the finger joints is still a work in progress, but very few microgesture studies currently use them [53].

µGlyph can also describe properties such as pressure, amplitude, and duration. µPoly cannot yet fully recognize the microgestures that take advantage of the symbols defining these properties, i.e. Hi, Md, and Lo for pressure, and Cl, Ha, and Op for amplitude. When the symbols are integrated into µPoly, the users will have to specify the threshold of sensor values corresponding to each symbol in the driver classes. Nevertheless, although pressure and amplitude symbols are not recognized, µPoly can already recognize changes, e.g. 20% → 50%, or magnitude of change, e.g. 30%, all expressed in percent. Duration is also implemented in µPoly but has not yet been integrated into the µGlyph editor.

Finally, µGlyph allows the definition of custom events by encapsulating a sequence of low-level events. These custom events are useful when recognizing a 2D drawing, e.g. a `circle` drawn in the air $\overset{\circ}{\circ}$. However, µPoly does not yet support them. This extension will require the addition of a new property to the low-level event, and encapsulation will therefore be managed at the driver level. Another solution envisioned to solve this problem, is to allow using µPoly recognizer references when defining a new µPoly recognizer (i.e., recursively using recognizers). This solution allow users to directly create their own encapsulated µGlyph expressions without changing µPoly source code.

Type	Strategy	What we did	Results	Ref
1	**Demonstration, individual instances, beyond descriptions**	Described µPoly Developed applications	✓ Complete functional description	section 4
			✓ Relevant application scenarios	section 5
			✓ Toolkit available with full documentation and video demonstrations	
			✓ Example applications (music player, key binder, VLC remote controller)	subsection 5.3
2	**Usage, early usage studies, feedback**	µPoly initial user tests	✓ Creating a driver: 2-3 days to fully understand µPoly's input format and create the adapted driver for their existing recognition system	section 6.1
			✓ Using µPoly without prior knowledge of µGlyph: ≈1h30 to write and test multiple microgestures with the official µGlyph tutorial as only base for learning	section 6.1
			✓ Receiving a microgesture in a third-party application: Successfull connection of µPoly to an automated home without any exterior help (controlling lights with microgestures, etc.)	section 6.1
3	**Performance**	Latency tests for a tap (≈310ms execution time)	✓ On a single computer (minimal network latency): µPoly latency (≈1ms) < network latency (≈42ms)	subsubsection 6.2.1
			✓ On multiple computers: ≈81 ms of network latency compared to an integrated commercial solution (Hololens 2)	subsubsection 6.2.2
			✓ Overall: µPoly latency < microgesture execution time	subsubsection 6.2.3
4	**Heuristic**	Built upon 5 properties from the literature $\mathbf{P_{visc}}$, $\mathbf{P_{ceil}}$, $\mathbf{P_{skill}}$, $\mathbf{P_{infra}}$ and $\mathbf{P_{proto}}$	✓ $\mathbf{P_{visc}}$: based on µGlyph standard for microgesture description	subsection 3.1
			✓ $\mathbf{P_{ceil}}$: µGlyph allows to combine elementary microgestures into more complex microgestures + µPoly supports the addition of new recognition systems	subsection 3.1
			✓ $\mathbf{P_{skill}}$: integrated library of commonly used microgestures + visual editor to avoid coding	subsection 3.1
			✓ $\mathbf{P_{infra}}$: event/subscriber websocket architecture	subsection 3.1
			✓ $\mathbf{P_{proto}}$: µPoly supports simulating microgestures	subsection 3.1

Table 3. Summary of our toolkit evaluation strategies and their results.

Overall, µPoly can, as of the writing of this paper, be used to recognize 97 of the 121 (80%) microgestures referenced in µDex.

7.2 Integrating calibration modules in µPoly

Many recognition systems need to be calibrated before they can be used. For example, in M[eye]cro [52], it is necessary to adapt the pressure threshold of the ad-hoc algorithm to each user. In fact, the range of sensor values changes according to hand size and glove stretch. With data gloves like Mocap[15], users have to perform certain hand poses to calibrate the device. For µPoly to work as a stand-alone microgesture recognition application, we would need to integrate a calibration UI into µPoly. Several approaches can be used. For example, by integrating an external web application using an iframe, or by offering the possibility of creating a customized UI component in µPoly's device tab.

7.3 Improving coordination of multiple recognition systems

Studies have combined information from several sensors to improve microgesture recognition [48]. With µPoly, this would be possible by creating a driver connected to several sensors.

Beyond coordination at the sensor level, µPoly can also coordinate multiple recognition systems. In the current version of µPoly, users can connect several recognition systems at the same time. The AR application in the previous section presents a use case in which two recognition systems are used, but each device recognizes different elementary microgestures. The coordination of several recognition systems therefore remains limited.

As µPoly communicates individually with each device, it is possible to determine the device from which the elementary microgesture or low-level event originates. This feature will enable us

[15] https://stretchsense.com/mocap-pro-fidelity-glove-2/

to filter elementary microgestures or low-level events according to device. With this feature, we envision two types of coordination to be implemented in µPoly:

- first, the ability to select the recognition systems to be used for each state machine. Users will have to select the recognition systems for each µGlyph description, using, for example, a list of checkboxes. This first level of control can be used to distinguish microgestures according to their recognition systems, for example to differentiate between left-handed and right-handed microgestures;
- second, the ability to select the recognition systems to be used for each transition of a state machine. This second level of control can be used to combine events from two different devices. A potential application would be bimanual microgestures where microgestures are performed by both hands in a complementary way.

7.4 Further Evaluation of µPoly

We already tested µPoly using all four strategies identified in the literature, as shown in Table 3. While in this study we focused on the core of µPoly, extended evaluations of the UI of µPoly will be carried out with users as part of future work. These evaluations will not impact the core principles of µPoly but will change the ergonomics to interact with the UI of µPoly, which, as a reminder, is not mandatory to use the µPoly toolkit. Finally, we envision the final evaluation of µPoly to be driven by the open-source release of the toolkit, enabling the community to contribute and further develop it. µPoly being part of a collaborative project involving academics and industry, developers not involved in µPoly development are already using the toolkit to develop an application offering microgesture-based interaction in autonomous cars, and another one for scrolling documents using microgestures in AR.

8 Conclusion

In this paper, we presented µPoly, the first toolkit designed to facilitate research and rapid prototyping of interactions based on microgestures. µPoly offers three major advantages: 1) easy switching between different microgesture recognition systems, 2) easy modification of the microgesture sets to be recognized, and 3) easy integration with third-party applications. To achieve these advantages, we base µPoly on the µGlyph microgesture notation [10]. With this toolkit, µGlyph gets one step closer to becoming a software engineering standard for describing microgesture events.

While µPoly will support and facilitate studies in the field of microgesture interaction, we hope that the core concepts of µPoly will be integrated into existing SDKs, such as the MRTK for the HoloLens 2 that already propose one elementary microgesture event: a pinch.

Acknowledgments

This work was partly supported by the French National Research Agency (ANR) project MIC (ANR-22-CE33-0017).

References

[1] [n.d.]. Hololens 2 Hand Tracking. https://learn.microsoft.com/en-us/windows/mixed-reality/mrtk-unity/mrtk2/features/input/hand-tracking?view=mrtkunity-2022-05. Accessed: 2024-03-26.

[2] [n.d.]. MediaPipe Hand Tracking. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker. Accessed: 2024-03-26.

[3] Caroline Appert and Michel Beaudouin-Lafon. 2006. SwingStates: adding state machines to the swing toolkit. In *Proceedings of the 19th Annual ACM Symposium on User Interface Software and Technology* (Montreux, Switzerland) *(UIST '06)*. Association for Computing Machinery, New York, NY, USA, 319–322. https://doi.org/10.1145/1166253.1166302

[4] Apple. 2024. *Use double tap to perform common actions on Apple Watch.* https://support.apple.com/en-gb/guide/watch/apdabb7b275c/watchos

[5] Frank Beruscha, Katharina Mueller, and Thorsten Sohnke. 2020. Eliciting Tangible and Gestural User Interactions with and on a Cooking Pan. In *Proceedings of the Conference on Mensch Und Computer* (Magdeburg, Germany) *(MuC '20)*. Association for Computing Machinery, New York, NY, USA, 399–408. https://doi.org/10.1145/3404983.3405516

[6] Roger Boldu, Alexandru Dancu, Denys J.C. Matthies, Pablo Gallego Cascón, Shanaka Ransir, and Suranga Nanayakkara. 2018. Thumb-In-Motion: Evaluating Thumb-to-Ring Microgestures for Athletic Activity. In *Proceedings of the 2018 ACM Symposium on Spatial User Interaction* (Berlin, Germany) *(SUI '18)*. Association for Computing Machinery, New York, NY, USA, 150–157. https://doi.org/10.1145/3267782.3267796

[7] Adrien Chaffangeon Caillet, Alix Goguey, and Laurence Nigay. 2025. Microgesture+ Grasp: A journey from human capabilities to interaction with microgestures. *International Journal of Human-Computer Studies* 195 (2025), 103398.

[8] Adrien Chaffangeon Caillet. 2024. *µDex: an online database for microgestures.* https://microglyph.imag.fr

[9] Adrien Chaffangeon Caillet, Alix Goguey, and Laurence Nigay. 2023. 3D Selection in Mixed Reality: Designing a Two-Phase Technique To Reduce Fatigue. In *2023 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. Sydney (Australia), Australia. https://hal.science/hal-04297966

[10] Adrien Chaffangeon Caillet, Alix Goguey, and Laurence Nigay. 2023. µGlyph: a Microgesture Notation. In *In Proceedings of the 2023 CHI Conference on Hu- man Factors in Computing Systems (CHI '23)* (Hamburg, Germany). New York, NY, USA, 28 pages. https://doi.org/10.1145/3544548.3580693

[11] Edwin Chan, Teddy Seyed, Wolfgang Stuerzlinger, Xing-Dong Yang, and Frank Maurer. 2016. User Elicitation on Single-Hand Microgestures. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems* (San Jose, California, USA) *(CHI '16)*. Association for Computing Machinery, New York, NY, USA, 3403–3414. https://doi.org/10.1145/2858036.2858589

[12] Liwei Chan, Yi Ling Chen, Chi Hao Hsieh, Rong Hao Liang, and Bing Yu Chen. 2015. Cyclopsring: Enabling whole-hand and context-aware interactions through a fisheye ring. *UIST 2015 - Proceedings of the 28th Annual ACM Symposium on User Interface Software and Technology* October (2015), 549–556. https://doi.org/10.1145/2807442.2807450

[13] Maximilian Christ, Nils Braun, Julius Neuffer, and Andreas Kempa-Liehr. 2018. Time Series FeatuRe Extraction on basis of Scalable Hypothesis tests (tsfresh – A Python package). *Neurocomputing* 307 (05 2018). https://doi.org/10.1016/j.neucom.2018.03.067

[14] Travis Deyle, Szabolcs Palinko, Erika Shehan Poole, and Thad Starner. 2007. Hambone: A Bio-Acoustic Gesture Interface. In *2007 11th IEEE International Symposium on Wearable Computers*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 3–10. https://doi.org/10.1109/ISWC.2007.4373768

[15] Rhett Dobinson, Marc Teyssier, Jürgen Steimle, and Bruno Fruchard. 2022. MicroPress: Detecting Pressure and Hover Distance in Thumb-to-Finger Interactions. In *Proceedings of the 2022 ACM Symposium on Spatial User Interaction* (Online, CA, USA) *(SUI '22)*. Association for Computing Machinery, New York, NY, USA, Article 4, 10 pages. https://doi.org/10.1145/3565970.3567698

[16] Christoph Endres, Tim Schwartz, and Christian A. Müller. 2011. Geremin": 2D Microgestures for Drivers Based on Electric Field Sensing. In *Proceedings of the 16th International Conference on Intelligent User Interfaces* (Palo Alto, CA, USA) *(IUI '11)*. Association for Computing Machinery, New York, NY, USA, 327–330. https://doi.org/10.1145/1943403.1943457

[17] Facebook. 2021. *Inside Facebook Reality Labs: Wrist-based interaction for the next computing platform.* https://tech.fb.com/inside-facebook-reality-labs-wrist-based-interaction-for-the-next-computing-platform/

[18] Gauthier Robert Jean Faisandaz, Alix Goguey, Christophe Jouffrais, and Laurence Nigay. 2022. Keep in Touch: Combining Touch Interaction with Thumb-to-Finger µGestures for People with Visual Impairment. In *Proceedings of the 2022 International Conference on Multimodal Interaction* (Bengaluru, India) *(ICMI '22)*. Association for Computing Machinery, New York, NY, USA, 105–116. https://doi.org/10.1145/3536221.3556589

[19] Gauthier Robert Jean Faisandaz, Alix Goguey, Christophe Jouffrais, and Laurence Nigay. 2023. µGeT: Multimodal eyes-free text selection technique combining touch interaction and microgestures. In *Proceedings of the 2023 International Conference on Multimodal Interaction* (Paris, France) *(ICMI '23)*. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3577190.3614131

[20] Nicholas Gillian and Joseph A. Paradiso. 2014. The Gesture Recognition Toolkit. *J. Mach. Learn. Res.* 15, 1 (jan 2014), 3483–3487.

[21] Saul Greenberg. 2007. Toolkits and interface creativity. *Multimedia Tools and Applications* 32, 2 (Feb. 2007), 139–159. https://doi.org/10.1007/s11042-006-0062-y

[22] Juan David Hincapié-Ramos, Xiang Guo, Paymahn Moghadasian, and Pourang Irani. 2014. Consumed endurance: a metric to quantify arm fatigue of mid-air interactions. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Toronto, Ontario, Canada) *(CHI '14)*. Association for Computing Machinery, New York, NY, USA, 1063–1072. https://doi.org/10.1145/2556288.2557130

[23] Yasha Iravantchi, Mayank Goel, and Chris Harrison. 2019. BeamBand: Hand Gesture Sensing with Ultrasonic Beamforming. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) *(CHI '19)*. Association for Computing Machinery, New York, NY, USA, 1–10. https://doi.org/10.1145/3290605.3300245

[24] Yasha Iravantchi, Yi Zhao, Kenrick Kin, and Alanson P. Sample. 2023. SAWSense: Using Surface Acoustic Waves for Surface-Bound Event Recognition. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 422, 18 pages. https://doi.org/10.1145/3544548.3580991

[25] Nikhita Joshi, Parastoo Abtahi, Raj Sodhi, Nitzan Bartov, Jackson Rushing, Christopher Collins, Daniel Vogel, and Michael Glueck. 2023. Transferable Microgestures Across Hand Posture and Location Constraints: Leveraging the Middle, Ring, and Pinky Fingers. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) *(UIST '23)*. Association for Computing Machinery, New York, NY, USA, Article 103, 17 pages. https://doi.org/10.1145/3586183.3606713

[26] D. Kammer, D. Henkens, C. Henzen, and R. Groh. 2015. Gesture Formalization for Multitouch. *Software: Practice and Experience* 45, 4 (apr 2015), 527–548. https://doi.org/10.1002/spe.2247

[27] Kenrick Kin, Björn Hartmann, Tony DeRose, and Maneesh Agrawala. 2012. Proton++: A Customizable Declarative Multitouch Framework. In *Proceedings of the 25th Annual ACM Symposium on User Interface Software and Technology* (Cambridge, Massachusetts, USA) *(UIST '12)*. Association for Computing Machinery, New York, NY, USA, 477–486. https://doi.org/10.1145/2380116.2380176

[28] Kenrick Kin, Björn Hartmann, Tony DeRose, and Maneesh Agrawala. 2012. Proton: Multitouch Gestures as Regular Expressions. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Austin, Texas, USA) *(CHI '12)*. Association for Computing Machinery, New York, NY, USA, 2885–2894. https://doi.org/10.1145/2207676.2208694

[29] Kenrick Kin, Chengde Wan, Ken Koh, Andrei Marin, Necati Cihan Camgöz, Yubo Zhang, Yujun Cai, Fedor Kovalev, Moshe Ben-Zacharia, Shannon Hoople, Marcos Nunes-Ueno, Mariel Sanchez-Rodriguez, Ayush Bhargava, Robert Wang, Eric Sauser, and Shugao Ma. 2024. STMG: A Machine Learning Microgesture Recognition System for Supporting Thumb-Based VR/AR Input. In *Proceedings of the CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 753, 15 pages. https://doi.org/10.1145/3613904.3642702

[30] David Ledo, Steven Houben, Jo Vermeulen, Nicolai Marquardt, Lora Oehlberg, and Saul Greenberg. 2018. Evaluation Strategies for HCI Toolkit Research. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems* (Montreal QC, Canada) *(CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–17. https://doi.org/10.1145/3173574.3173610

[31] DoYoung Lee, SooHwan Lee, and Ian Oakley. 2020. Nailz: Sensing Hand Input with Touch Sensitive Nails. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems (CHI '20)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3313831.3376778

[32] Jaime Lien, Nicholas Gillian, M. Emre Karagozler, Patrick Amihood, Carsten Schwesig, Erik Olson, Hakim Raja, and Ivan Poupyrev. 2016. Soli: Ubiquitous Gesture Sensing with Millimeter Wave Radar. *ACM Trans. Graph.* 35, 4, Article 142 (jul 2016), 19 pages. https://doi.org/10.1145/2897824.2925953

[33] Yi Liu, Hongying Meng, Mohammad Rafiq Swash, Yona Falinie A. Gaus, and Rui Qin. 2018. Holoscopic 3D Micro-Gesture Database for Wearable Device Interaction. In *2018 13th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2018)*. Institute of Electrical and Electronics Engineers, New York, NY, USA, 802–807. https://doi.org/10.1109/FG.2018.00129

[34] Wendy E. Mackay. 1998. Augmented Reality: Linking Real and Virtual Worlds: A New Paradigm for Interacting with Computers. In *Proceedings of the Working Conference on Advanced Visual Interfaces* (L'Aquila, Italy) *(AVI '98)*. Association for Computing Machinery, New York, NY, USA, 13–21. https://doi.org/10.1145/948496.948498

[35] Nicolai Marquardt, Johannes Kiemer, David Ledo, Sebastian Boring, and Saul Greenberg. 2011. Designing user-, hand-, and handpart-aware tabletop interactions with the TouchID toolkit. In *Proceedings of the ACM International Conference on Interactive Tabletops and Surfaces* (Kobe, Japan) *(ITS '11)*. Association for Computing Machinery, New York, NY, USA, 21–30. https://doi.org/10.1145/2076354.2076358

[36] George B. Mo, John J Dudley, and Per Ola Kristensson. 2021. Gesture Knitter: A Hand Gesture Design Tool for Head-Mounted Mixed Reality Applications. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 291, 13 pages. https://doi.org/10.1145/3411764.3445766

[37] Brad Myers, Scott E. Hudson, and Randy Pausch. 2000. Past, Present, and Future of User Interface Software Tools. *ACM Trans. Comput.-Hum. Interact.* 7, 1 (mar 2000), 3–28. https://doi.org/10.1145/344949.344959

[38] Yanghee Nam, Nwangyun Wohn, and Hyung Lee-Kwang. 1999. Modeling and recognition of hand gesture using colored Petri nets. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 29, 5 (1999), 514–521. https://doi.org/10.1109/3468.784178

[39] William M. Newman. 1968. A System for Interactive Graphical Programming. In *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference* (Atlantic City, New Jersey) *(AFIPS '68 (Spring))*. Association for Computing Machinery, New York, NY, USA, 47–54. https://doi.org/10.1145/1468075.1468083

[40] Dan R. Olsen. 2007. Evaluating User Interface Systems Research. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology* (Newport, Rhode Island, USA) *(UIST '07)*. Association for Computing Machinery, New York, NY, USA, 251–258. https://doi.org/10.1145/1294211.1294256

[41] Carole Plasson, Renaud Blanch, and Laurence Nigay. 2022. Selection Techniques for 3D Extended Desktop Workstation with AR HMD. In *2022 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. 460–469. https://doi.org/10.1109/ISMAR55827.2022.00062

[42] T. Saponas, Desney Tan, Dan Morris, and Ravin Balakrishnan. 2008. Demonstrating the feasibility of using forearm electromyography for muscle-computer interfaces. *Conference on Human Factors in Computing Systems - Proceedings*, 515–524. https://doi.org/10.1145/1357054.1357138

[43] T. Scott Saponas, Desney S. Tan, Dan Morris, Ravin Balakrishnan, Jim Turner, and James A. Landay. 2009. Enabling Always-Available Input with Muscle-Computer Interfaces. In *Proceedings of the 22nd Annual ACM Symposium on User Interface Software and Technology* (Victoria, BC, Canada) *(UIST '09)*. Association for Computing Machinery, New York, NY, USA, 167–176. https://doi.org/10.1145/1622176.1622208

[44] Teddy Seyed, Alaa Azazi, Edwin Chan, Yuxi Wang, and Frank Maurer. 2015. SoD-Toolkit: A Toolkit for Interactively Prototyping and Developing Multi-Sensor, Multi-Device Environments. In *Proceedings of the 2015 International Conference on Interactive Tabletops & Surfaces* (Madeira, Portugal) *(ITS '15)*. Association for Computing Machinery, New York, NY, USA, 171–180. https://doi.org/10.1145/2817721.2817750

[45] Adwait Sharma, Michael A. Hedderich, Divyanshu Bhardwaj, Bruno Fruchard, Jess McIntosh, Aditya Shekhar Nittala, Dietrich Klakow, Daniel Ashbrook, and Jürgen Steimle. 2021. SoloFinger: Robust Microgestures While Grasping Everyday Objects. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (Yokohama, Japan) *(CHI '21)*. Association for Computing Machinery, New York, NY, USA, Article 744, 15 pages. https://doi.org/10.1145/3411764.3445197

[46] Adwait Sharma, Joan Sol Roo, and Jürgen Steimle. 2019. Grasping Microgestures: Eliciting Single-Hand Microgestures for Handheld Objects. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) *(CHI '19)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3290605.3300632

[47] Adwait Sharma, Christina Salchow-Hömmen, Vimal Suresh Mollyn, Aditya Shekhar Nittala, Michael A. Hedderich, Marion Koelle, Thomas Seel, and Jürgen Steimle. 2023. SparseIMU: Computational Design of Sparse IMU Layouts for Sensing Fine-grained Finger Microgestures. *ACM Transactions on Computer-Human Interaction* 30, 3 (jun 2023), 1–40. https://doi.org/10.1145/3569894

[48] Mohamed Soliman, Franziska Mueller, Lena Hegemann, Joan Sol Roo, Christian Theobalt, and Jürgen Steimle. 2018. FingerInput: Capturing Expressive Single-Hand Thumb-to-Finger Microgestures. In *Proceedings of the 2018 ACM International Conference on Interactive Surfaces and Spaces* (Tokyo, Japan) *(ISS '18)*. Association for Computing Machinery, New York, NY, USA, 177–187. https://doi.org/10.1145/3279778.3279799

[49] Hyunyoung Song, Hrvoje Benko, Francois Guimbretiere, Shahram Izadi, Xiang Cao, and Ken Hinckley. 2011. Grips and Gestures on a Multi-Touch Pen. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Vancouver, BC, Canada) *(CHI '11)*. Association for Computing Machinery, New York, NY, USA, 1323–1332. https://doi.org/10.1145/1978942.1979138

[50] Yanke Tan, Sang Ho Yoon, and Karthik Ramani. 2017. BikeGesture: User Elicitation and Performance of Micro Hand Gesture as Input for Cycling. In *Proceedings of the 2017 CHI Conference Extended Abstracts on Human Factors in Computing Systems (CHI EA '17)*. Association for Computing Machinery, New York, NY, USA, 2147–2154. https://doi.org/10.1145/3027063.3053075

[51] Anandghan Waghmare, Youssef Ben Taleb, Ishan Chatterjee, Arjun Narendra, and Shwetak Patel. 2023. Z-Ring: Single-Point Bio-Impedance Sensing for Gesture, Touch, Object and User Recognition. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing

Machinery, New York, NY, USA, Article 150, 18 pages. https://doi.org/10.1145/3544548.3581422

[52] Jérémy Wambecke, Alix Goguey, Laurence Nigay, Lauren Dargent, Daniel Hauret, Stéphanie Lafon, and Jean-Samuel Louis de Visme. 2021. M[Eye]Cro: Eye-Gaze+Microgestures for Multitasking and Interruptions. *Proc. ACM Hum.-Comput. Interact.* 5, EICS, Article 210 (May 2021), 22 pages. https://doi.org/10.1145/3461732

[53] Way, David. 2014. *EMGRIE: Ergonomic Microgesture Recognition and Interaction Evaluation, A Case Study.* Technical Report 2000. Massachusetts Institute of Technology. 1–195 pages.

[54] Eric Whitmire, Mohit Jain, Divye Jain, Greg Nelson, Ravi Karkar, Shwetak Patel, and Mayank Goel. 2017. DigiTouch. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 1, 3 (2017), 1–21. https://doi.org/10.1145/3130978

[55] Katrin Wolf, Anja Naumann, Michael Rohs, and Jörg Müller. 2011. A taxonomy of microinteractions: Defining microgestures based on ergonomic and scenario-Dependent requirements. *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 6946 LNCS, PART 1 (2011), 559–575. https://doi.org/10.1007/978-3-642-23774-4_45

[56] Zheer Xu, Pui Chung Wong, Jun Gong, Te Yen Wu, Aditya Shekhar Nittala, Xiaojun Bi, Jürgen Steimle, Hongbo Fu, Kening Zhu, and Xing Dong Yang. 2019. TipText: Eyes-free text entry on a fingertip keyboard. *UIST 2019 - Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology* (2019), 883–889. https://doi.org/10.1145/3332165.3347865

A Code snippets from UPoly documentation

This section includes code snippets for new sensing devices and application examples. Both snippets are extracted from the µPoly documentation [16] , in particular the *Create your own driver* page[17] and the *Create your own external application* page[18]

```
const Device = require("../Device.js");
const RawDataEvent = require("../../RawDataEvent.js");
////////////////////////////// READ ME ////////////////////////////////
// This is an example of an empty device, that you can use to create your own device.
// This Class is not supposed to be instanciated, but to be used as a TEMPLATE to create your own device.
// Some generic methode FooProtocol, barMethod, have to be replaced by your own system.
// Override methods that need to be overrided.

class EmptySerialDevice extends Device {
    static protocol = Device.PROTOCOL.SERIAL;
    //add a constructor with em argument
    constructor(index, am, port = null) {
        super(index, "EmptyArduinoDevice", am);
        // this.port = port; //if you use a standard protocol,and that you define the static protocol below. User will be able to choose the port in the UI
        this.description = "This is an empty device, use it as a template to create your own device"
        this.openPort(port); //try to open the serial port, even if it's null
    }

    openPort(port) {
        // Create a port with error callback
        this.port = port;

        if (port == null || port == undefined) {
            console.error("No port defined");
            return;
        }

        //Helper function that create automatically device
        //Regarging protocol and port. Set automatically this.device
        let promiseDevice = this.createDevice()

        promiseDevice.then(() => {
            // Event when data is received, Only is device exist
            this.device.on('data', (data) => {
                // Manage your data here
                this.barMethod(data);
            });
        });
    }

    //override this method
```

[16] velvet-noise.notion.site/Documentation-Poly-0912a482fdcf4374b4ea7e4cd3205e0d

[17] How to create a driver: https://velvet-noise.notion.site/Create-your-own-Device-c5b010075b164cb2a09eace21680bf06

[18] How to connect an external application: https://velvet-noise.notion.site/Create-your-own-external-application-bbbd38ca087d42a79c59e32f822c8367

```
    restartConnection() {
        // This method is called when the connection is lost
        if (!this.isOpen) {
            this.openPort(this.port);
        } else {
            this.close();
            setTimeout(() => {
                this.openPort(this.port);
            }, 1000);
        }
    }

    //override close method
    close() {
        // When you are done using the serial port, you can close it like this:
        if (this.device?.isOpen) {
            this.device.close((err) => {
                if (err) {
                    console.error('Error closing the serial port:', err);
                } else {
                    console.log('Serial port closed successfully.');
                }
            });
        }
        //clear this.device as null
        this.device = null;
    }

    barMethod(data) {
        //Process Data first in the way you want.

        // OPTIONNAL : have a look at a realtime view for a sensor for debug
        this.graphUI(value, channel)
        // OPTIONNAL : had a texte message to the console for debug
        this.consoleUI(message)

        // Send rawDataEvent if needed
        let rde = new RawDataEvent();
        // Modify RawDataEvent
        rde.actuator = [[rde.FINGER.THUMB]];
        rde.action = rde.ACTION.FLEXION
        rde.context = [rde.CONTEXT.AIR, rde.CONTEXT.AIR]
        //Emit this rawdataevent
        this.emit(rde);
    }
}

module.exports = EmptySerialDevice;
```

Snippet 1. Skeleton class to create a driver for a new sensing device connected through the serialport of the computer.

```python
import socketio, keyboard

sio = socketio.Client()

@sio.event
def connect():
    print('connection established')

@sio.event
def microgesture(data):
    # "microgesture" websocket event
    name = data['name']
    timestamp = data['timestamp']

    if name == "TAP":
        print("tap detected")
        keyboard.press_and_release('space')
    elif name == "DOUBLE_TAP":
        print("double_tap detected")
        keyboard.press_and_release('ctrl+s')
    else:
        print("unknown gesture detected : ", name)

@sio.event
def disconnect():
    print('disconnected from server')

while not sio.connected:
    try:
        sio.connect('http://localhost:9994')
    except socketio.exceptions.ConnectionError:
        pass
sio.wait()
```

Snippet 2. Example of Python program binding µPoly issued microgestures to keyboard keys

```
{
  "type": "microgesture",
  "name": "name of the microgesture",
  "data": null, // wip
  "deviceId": deviceId,
  "timestamp": 1709643996573,//Format Date().now
  "raw": {} // JSON version of the microgesture.
}
```

Snippet 3. Example of a microgesture event received by an external application.